Smart Contract Assessment

MO

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Aug 10, 2026 - Aug 11, 2026

SOURCE CODE
mgusd-minter-gateway

ASSESSED COMMIT ID
79b2a53

SEVERITY BREAKDOWN

3	2	1	0
All Findings	Solved	Acknowledged	Risk Accepted

Introduction

`M0` engaged Halborn to conduct a differential security assessment of the **Stellar Minter Gateway** on August 11th, 2026. The review covered only the changes between commits `e2afb7a` and `79b2a53`.

The scoped changes introduce user onboarding, persistent `Onboarded` and `BlockListed` policy records, new role separation, batch onboarding, and refactored compliance operations. Unchanged components were reviewed only when necessary to understand the security effects of these changes.

Assessment Summary

The team at Halborn assigned a full-time security engineer to verify the security of the scoped smart contract changes. The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.

The purpose of this differential assessment was to:

- Ensure that the newly introduced and modified smart contract functions operate as intended.
- Identify security issues and regressions introduced by the scoped code changes.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which have been completely addressed by the `M0` team. The main ones were the following:

- `Verify and reconcile current SAC authorization when re-onboarding previously approved accounts.`
- `Choose between trustline-dependent blocking with off-chain screening and persistent pre-trustline on-chain holds.`
- `Document block_user as limited SAC deauthorization, including that pre-existing offers and liquidity-pool liabilities may still settle after a block.`

Test Approach and Methodology

Halborn performed a combination of manual and automated security testing to balance efficiency, timeliness, practicality, and accuracy with respect to the differential scope of this assessment. Manual testing was emphasized to uncover flaws in logic, state synchronization, trustline lifecycle handling, and cross-contract interaction, while automated tools supported the detection of unsafe coding patterns and regressions.

The following phases and associated approaches were used during the assessment:

- Mapping the scoped diff and commit history.
- Reviewing all modified contract and SDK components.
- Verifying the new onboarder and compliance roles.
- Analyzing `Onboarded` and `BlockListed` storage.
- Tracing SAC authorization and classic trustline behavior.
- Reviewing single-user and batch compliance operations.
- Testing missing, removed, and recreated trustlines.
- Validating transaction atomicity and batch rollback.
- Running the complete test suite and static analysis.

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY

(a) Repository: mgusd-minter-gateway

(b) Assessed Commit ID: 79b2a53951ea03325efb745e7fdd504e9559ca62

(c) Items in scope:

- contracts/mintergateway/src 8 files
  - block_list.rs Rust
  - constants.rs Rust
  - contract.rs Rust
  - errors.rs Rust
  - events.rs Rust
  - lib.rs Rust
  - roles.rs Rust

storage_types.rs Rust

REMEDIATION COMMIT ID:

- 3f36ee6bcea59e477e501eebb89d3ed9f6516552
- 898ebacfeea9a6b4a8a3aed256e322c4cb175beb

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Previously Onboarded Accounts Are Not Reauthorized After Recreating Their Trustline	Informational	1.7	SOLVED 08/14/2026
HAL-002	Contract-Level Compliance Holds Cannot Be Recorded Before Trustline Creation	Informational	—	ACKNOWLEDGED 08/14/2026
HAL-003	Blocking Documentation Does Not Describe the Preservation of Existing Stellar Liabilities	Informational	—	SOLVED 08/14/2026

Findings & Tech Details

HAL-001 SOLVED

Previously Onboarded Accounts Are Not Reauthorized After Recreating Their Trustline

● Informational · 1.7 AO:A/AC:L/AX:M/R:P/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The scoped changes introduce a persistent and monotonic `Onboarded` record that identifies accounts previously approved by an onboarder. This record is independent from the lifecycle of the holder's classic Stellar trustline.

A holder with zero MGUSD balance and no liabilities can remove and later recreate the trustline. Under `AUTH_REQUIRED`, the recreated trustline is unauthorized, while `Onboarded(user)` remains set in the gateway.

Both `onboard_user` and `batch_onboard_users` treat the historical record as proof of current authorization and return success without consulting the SAC. `unblock_user` also performs no update because the account is not on `BlockListed`. The holder therefore remains unable to use MGUSD despite a successful onboarding response.

The state can be recovered through `block_user` followed by `unblock_user`, but this undocumented sequence requires coordination between two operational roles.

Code Location

`onboard_user` from `contracts/mintergateway/src/contract.rs`, where the historical `Onboarded` record causes an early successful return without checking SAC authorization:

RUST 332–355

```rust
pub fn onboard_user(
    e: Env,
    user: Address,
```

```
    operator: Address,
) -> Result<(), MinterGatewayError> {
    require_onboarder(&e, &operator)?;
    extend_instance_ttl(&e);

    if is_on_block_list(&e, &user) {
        return Err(MinterGatewayError::UserBlockedError);
    }

    if is_onboarded(&e, &user) {
        return Ok(());
    }

    insert_onboarded(&e, &user);

    let sac_addr = read_sac_token(&e);
    token::StellarAssetClient::new(&e, &sac_addr).set_authorized(&user, &true);

    emit_user_onboarded(&e, &user);
    Ok(())
}
```

`batch_onboard_users` from `contracts/mintergateway/src/contract.rs`, where previously onboarded addresses are skipped without reconciling their current SAC authorization state:

RUST 361–372

```
for user in users.iter() {
    if is_on_block_list(&e, &user) {
        blocked.push_back(user);
        continue;
    }
    if is_onboarded(&e, &user) {
        continue;
    }
    insert_onboarded(&e, &user);
    sac_client.set_authorized(&user, &true);
    emit_user_onboarded(&e, &user);
}
```

Impact

A legitimate holder who removes and later recreates an empty MGUSD trustline cannot be reactivated through the onboarding interface. The onboarding transaction reports success, but subsequent transfers and mint operations continue to fail because the trustline remains unauthorized.

The condition does not enable unauthorized transfers or bypass a compliance hold. Its impact is limited to loss of availability for the affected account and additional operational coordination until a block-then-unblock recovery is performed.

RECOMMENDATION

It is recommended to retain `Onboarded` as a monotonic record of historical approval while removing the assumption that its presence proves current SAC authorization.

When `onboard_user` finds an existing `Onboarded` record and no `BlockListed` record, it should query the SAC authorization state. If the trustline exists and is already authorized, the function can return as an idempotent no-op. If the trustline exists but is unauthorized, the onboarder should reapply `set_authorized(user, true)`. If no trustline exists, the function should return the typed `NoTrustline` error rather than reporting success.

The same reconciliation should be applied per address in `batch_onboard_users`. A distinct reauthorization event is recommended so operators can distinguish a historical onboarding no-op from an actual SAC state repair.

REMEDIATION COMMENT

Addressed in: 3f36ee6bcea59e477e501eebb89d3ed9f6516552

SOLVED: The `M0` team addressed this issue by updating `onboard_user` to reauthorize previously onboarded accounts whose recreated trustline is unauthorized and, in follow-up commit `6dfacd13`, applying equivalent reconciliation to `batch_onboard_users`.

HAL-002 ACKNOWLEDGED

Contract-Level Compliance Holds Cannot Be Recorded Before Trustline Creation

● Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

`block_user` writes the new persistent `BlockListed` record before calling SAC `set_authorized(user, false)`. For a classic Stellar account without an MGUSD trustline, the SAC call traps and Soroban atomicity reverts the entire invocation, including the preceding block-list write and event.

As a result, the contract cannot record a preventive compliance hold before the target creates a trustline. The same behavior affects `batch_block_users`: one address without a trustline reverts all block operations in the batch. Directed tests confirmed both cases.

This is an informational design limitation rather than an authorization bypass.

Code Location

`block_user` from `contracts/mintergateway/src/contract.rs`, where a failed SAC authorization update also reverts the newly written compliance record:

RUST 284–299

```rust
284 pub fn block_user(e: Env, user: Address, operator: Address) -> Result<(), MinterGatewayError> {
    require_block_operator(&e, &operator)?;
285     extend_instance_ttl(&e);
286
287     if is_on_block_list(&e, &user) {
288         return Ok(());
289     }
290
291     add_to_block_list(&e, &user);
292
293     let sac_addr = read_sac_token(&e);
294     token::StellarAssetClient::new(&e, &sac_addr).set_authorized(&user, &false);
295
```

```
296     emit_user_blocked(&e, &user);
297     Ok(())
298 }
299
```

`batch_block_users` from `contracts/mintergateway/src/contract.rs`, where all addresses share the same atomic invocation and one failed SAC call reverts the complete batch:

RUST 396–403

```rust
396 for user in users.iter() {
397     if is_on_block_list(&e, &user) {
398         continue;
399     }
400     add_to_block_list(&e, &user);
401     sac_client.set_authorized(&user, &false);
402     emit_user_blocked(&e, &user);
403 }
```

Impact

There is no direct security impact. The current behavior reflects a valid design choice in which an address can only be added to `BlockListed` when its trustline can also be deauthorized.

The finding only highlights an alternative design preference: allowing preventive block-list entries to be recorded independently of the address's current trustline state.

RECOMMENDATION

It is recommended to select and document one of two valid compliance models: retain the current trustline-dependent design and enforce pre-trustline screening off-chain, or decouple policy from SAC enforcement by allowing `BlockListed` to persist before trustline creation and applying deauthorization once the trustline exists.

REMEDIATION COMMENT

ACKNOWLEDGED: The `M0` team acknowledged this finding and elected to retain the current trustline-dependent compliance model. Accounts without an MGUSD trustline cannot transact and remain unauthorized under `AUTH_REQUIRED`, while persisting `BlockListed` only after successful SAC deauthorization preserves the invariant that every recorded hold is immediately enforceable.

HAL-003 SOLVED

Blocking Documentation Does Not Describe the Preservation of Existing Stellar Liabilities

● Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The gateway documentation describes `block_user` as removing an account from the allowlist and preventing it from sending or receiving MGUSD. The implementation delegates this operation to the Stellar Asset Contract by calling `set_authorized(user, false)` and subsequently reports the account as blocked through the inverse of SAC authorization.

For a classic Stellar trustline, this operation intentionally applies limited authorization rather than a hard freeze. The SAC clears `AuthorizedFlag` and sets `AuthorizedToMaintainLiabilitiesFlag`. Under the standard Stellar semantics, the account cannot initiate new payments, transfers, offers, or liquidity-pool deposits, but pre-existing liabilities are preserved: existing offers may remain active or be cancelled, and existing liquidity-pool positions may be withdrawn.

This behavior is intentional. It permits previously established positions to be reduced or settled without creating new liabilities and avoids disrupting counterparties or third-party protocols that depend on the orderly completion of already-open positions. Therefore, preserving existing liabilities is not considered a bypass of the block control under the intended operating model.

The remaining concern is documentary. The current description of a blocked account can be interpreted as complete immobilization, while the boolean `blocked(account)` interface does not distinguish a fully unauthorized classic trustline from one authorized only to maintain liabilities. Operators and monitoring systems may consequently assume stronger freeze semantics than the SAC provides.

RECOMMENDATION

It is recommended to document `block_user` as applying limited SAC deauthorization rather than complete immobilization for classic trustlines. The documentation should explicitly distinguish the post-block behavior:

- New payments, transfers, offers, and liquidity-pool deposits are prohibited.

- Existing offers may remain executable or be cancelled.
- Existing liquidity-pool positions may be withdrawn.
- `blocked(account)` reports the SAC authorization state but does not expose the underlying `AuthorizedToMaintainLiabilitiesFlag` distinction.

REFERENCES

Stellar documentation — Asset design and authorization flags

Stellar documentation — Revoking authorization through the SAC

REMEDIATION COMMENT

Addressed in: 898ebacfeea9a6b4a8a3aed256e322c4cb175beb

SOLVED: The `M0` team has solved this issue by documenting that `block_user` applies SAC deauthorization without cancelling standing offers or withdrawing liquidity-pool deposits, and that pre-existing offers may continue executing after the holder is blocked.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.